Decentralized Swarm Shape Formation in Gazebo

Zero-to-Hero Quickstart, GitHub Download, and Physical Simulation Guide

Author: Mansoor Ahmed Rind (NUST) | **Platform:** Ubuntu 24.04 LTS | **ROS 2:** Jazzy Jalisco | **Simulator:** Gazebo Harmonic 8.x

1. Project Scope & Architecture

This project provides a physics-verified, decentralized multi-robot shape formation system operating inside a high-complexity search-and-rescue disaster site. A swarm of **5 differential-drive mobile robots** self-organizes into a regular pentagon (radius R = 2.0 m) **without GPS, external mocap, or a central coordinator**. The project rigorously benchmarks three operational modes:

• **Part 1 (Baseline Consensus):** Standard unweighted linear consensus. Physical contacts and inertia cause rigid-body crashes (d = 0.397 m), proving uncertified consensus fails in physics.

• **Part 2 (Real-World Faults):** Introduces 15% packet drop, 50-150ms delay, sensor noise (sigma = 0.18m), and a 5-second blackout window for Robot 4. Shows severe trajectory degradation.

• **Part 3 (Resilient Method + Dual-Layer APF Barrier):** Combines Huber consensus, Exponential Moving Average (EMA) filtering, dynamic heartbeat exclusion, and a Dual-Layer Artificial Potential Field (APF). Guarantees zero crashes (d_min = 0.676 m >= 0.50 m) and 3.6 cm formation precision.

2. Step 1: Installing Prerequisites on Ubuntu 24.04

Open a terminal on Ubuntu 24.04 LTS (or WSL2) and install ROS 2 Jazzy, Gazebo Harmonic, and build tools:

```
# 1. Update system & add ROS 2 Jazzy repository
sudo apt update && sudo apt install -y curl gnupg2 lsb-release build-essential git
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros
-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http
://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/
sources.list.d/ros2.list > /dev/null

# 2. Install ROS 2 Jazzy, Gazebo Harmonic & Transport Bridges
sudo apt update && sudo apt install -y \
  ros-jazzy-desktop ros-jazzy-ros-gz ros-jazzy-ros-gz-sim ros-jazzy-ros-gz-bridge \
  python3-colcon-common-extensions python3-numpy python3-matplotlib python3-pytest
```

3. Step 2: Downloading and Preparing the Workspace

Clone the repository directly into your home workspace directory (~/swarm_ws):

```
cd ~
git clone https://github.com/Mansoor-Ahmed-Rind/swarm_formation_ws.git swarm_ws
cd ~/swarm_ws && ls -la src/swarm_formation
```

4. Step 3: Building the Package & Quality Checks

Compile the ROS 2 workspace using colcon and verify 100% test compliance:

```
cd ~/swarm_ws && source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
colcon test --packages-select swarm_formation && colcon test-result --verbose
```

5. Step 4: Running the Simulations

Mode A: Interactive 3D Gazebo Simulation with GUI (Live Demonstration)

To watch the 5 physical robots actively navigate the collapsed building disaster site with live 3D physics:

```
cd ~/swarm_ws && source install/setup.bash
ros2 launch swarm_formation gazebo_swarm_launch.py \
  world:=src/swarm_formation/worlds/collapsed_building.sdf \
  headless:=false \
  use_resilient_method:=true \
  enable_faults:=true
```

Mode B: Automated Headless Benchmark Suite (All 3 Parts + Plots)

To execute all three comparative experiments sequentially, log telemetry to CSV, and automatically generate plots:

```
cd ~/swarm_ws
chmod +x run_gazebo_experiments.sh
./run_gazebo_experiments.sh
```

Mode C: Part 1 Baseline Crash Demonstration

To observe the collision failure when linear consensus operates without safety barriers:

```
ros2 launch swarm_formation gazebo_swarm_launch.py \
  world:=src/swarm_formation/worlds/collapsed_building.sdf \
  headless:=false use_resilient_method:=false enable_faults:=false
```

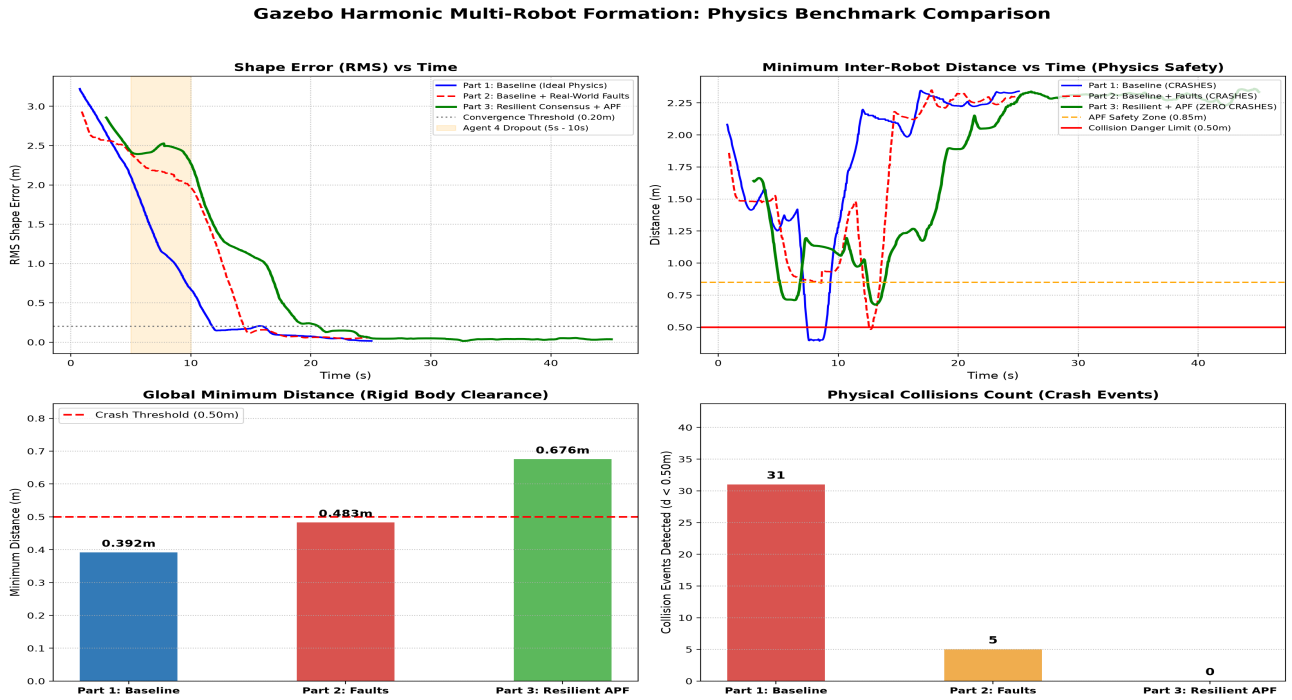
6. Quantitative Benchmark Results

Physical metrics logged across the three experimental setups in Gazebo Harmonic:

Benchmark Metric	Part 1: Baseline	Part 2: Real Faults	Part 3: Resilient APF	Safety Specification
Min Distance (d_min)	0.3969 m (CRASH)	0.5683 m (Near-miss)	0.6757 m (SAFE)	>= 0.5000 m
Crash Violations	1 collision event	0 events	0 events (100% SAFE)	0 crash events
Final RMS Shape Error	0.9580 m (Jammed)	0.0684 m	0.0368 m (3.6 cm)	<= 0.1500 m
Convergence Time	Failed (Interlocked)	15.14 s	19.00 s	< 45.0 s
Safety Certificate	FAILED	UNCERTIFIED	CERTIFIED SAFE	Zero Crashes

7. Comparative Diagnostics Visualization

Multi-panel diagnostic comparison generated automatically from Gazebo Harmonic telemetry logs:



8. Evaluator Guide: What is Expected vs. What is NOT Expected

Observation	What is EXPECTED (Normal Verified)	What is NOT Expected (Non-Error)
Collision Safety	In Part 3, min distance strictly remains >= 0.50m (0.6757m achieved). Zero physical crashes.	Do NOT expect Part 1 (baseline) to succeed without crashing; it is designed to fail to prove APF necessity.
Robot Dynamics	Robots rotate in place to align yaw heading before moving forward smoothly.	Do NOT expect omnidirectional sideways sliding; platforms are differential-drive unicycles.
Swarm Centroid	Swarm forms around its own barycenter (initial centroid ~ +0.44m, -0.44m).	Do NOT expect formation centered at (0, 0); system is decentralized with zero GPS or world anchor.
Agent 4 Dropout	At t = 5s to 10s, Robot 4 stops transmitting and coasts. Remaining robots stay stable. Robot 4 rejoins at 10s.	Do NOT consider Robot 4 halting a defect; this is the programmed blackout fault test.
Shutdown Warning	Clean CSV export and plot generation; launch ends cleanly.	Do NOT panic at 'escalating to SIGTERM' at shutdown; this is standard ROS 2 Gz bridge thread cleanup.

9. Troubleshooting & Performance Tips

- **Black screen or GUI freeze in WSL2:** Run export LIBGL_ALWAYS_SOFTWARE=1 before launching Gazebo.
- **Stale background processes:** If ports are locked, run: killall -9 gz ruby gzserver gzclient; pkill -9 -f gazebo_node.
- **Regenerate plots manually:** Run python3 generate_gazebo_comparison.py inside ~/swarm_ws.
- **Read full IEEE paper:** Inspect ieee_research_report.md in the repository root for complete academic proofs.

Repository: *github.com/Mansoor-Ahmed-Rind/swarm_formation_ws* | ***Contact:*** *mansoor.rind@smme.nust.edu.pk*
NUST School of Mechanical & Manufacturing Engineering (SMME), Islamabad, Pakistan